

Swift / ...

Protocol

CVarArg

A type whose instances can be encoded, and appropriately passed, as elements of a C `va_list`.

iOS 8.0+ | iPadOS 8.0+ | Mac Catalyst 13.0+ | macOS 10.10+ | tvOS 9.0+ | visionOS 1.0+ |
watchOS 2.0+

`protocol` CVarArg

Mentioned in

 [Using Imported C Functions in Swift](#)

Overview

You use this protocol to present a native Swift interface to a C “varargs” API. For example, a program can import a C API like the one defined here:

```
int c_api(int, va_list arguments)
```

To create a wrapper for the `c_api` function, write a function that takes `CVarArg` arguments, and then call the imported C function using the `withVaList(_:_:_)` function:

```
func swiftAPI(_ x: Int, arguments: CVarArg...) -> Int {  
    return withVaList(arguments) { c_api(x, $0) }  
}
```

Swift only imports C variadic functions that use a `va_list` for their arguments. C functions that use the `...` syntax for variadic arguments are not imported, and therefore can’t be called using `CVarArg` arguments.

If you need to pass an optional pointer as a `CVarArg` argument, use the `Int(bit Pattern:)` initializer to interpret the optional pointer as an `Int` value, which has the same C variadic calling conventions as a pointer on all supported platforms.

Note

Declaring conformance to the `CVarArg` protocol for types defined outside the standard library is not supported.

Relationships